C-CAT
SECTION B
PREP GUIDE

Operating Systems

Complete Revision Notes

Topics 1–9 • Zero to Hero • B.Tech CSE

Green & Orange Edition

Personalised coaching notes — All 9 OS chapters covered with exam traps highlighted
Designed for 1–2 hour revision • Exam-ready format • Phase 1: OS Complete

TABLE OF CONTENTS

TOPIC 01

What is an OS + Booting Process

■ WHAT IS AN OPERATING SYSTEM?

An **Operating System (OS)** is a **system software** (not an app — it IS the platform that apps run on) that manages all hardware and provides services to user programs. It sits between hardware and user applications, acting as a **resource manager and protector**.

■ Hostel Warden Analogy

- Hostel = Computer | Rooms = RAM | Kitchen = CPU | Students = Programs
- Warden (OS) decides: who gets a room, who uses the kitchen, who waits outside
- Warden ensures Student A can't barge into Student B's room and mess with their stuff
- Without a warden → chaos, fights, wasted resources, system crashes

■ FIVE MAIN JOBS OF THE OS

- **Process Management** — Runs, schedules, and switches between multiple programs — decides who gets the CPU and when
- **Memory Management** — Allocates portions of RAM to each process and frees it when done
- **File Management** — Organises data into named files and directories on the disk
- **Device Management** — Controls hardware devices — keyboard, printer, disk, screen
- **Security / Protection** — Prevents processes from accessing each other's memory or crashing the system

■ BOOTING PROCESS — STEP BY STEP

When you press the power button, RAM is completely empty — the OS isn't there yet. Booting is the process of loading the OS from disk into RAM so it can take control.

① Power ON — electricity flows to all components
↓
② BIOS / UEFI runs — tiny program permanently stored on motherboard chip (not disk!)
↓
③ POST (Power-On Self-Test) — checks RAM, keyboard, disk are working. Failure = beeps / error
↓
④ Bootloader located on disk and loaded into RAM by BIOS/UEFI
↓
⑤ Bootloader loads the OS Kernel into RAM — then STOPS (its job is done!)
↓
⑥ Kernel initialises memory manager, device drivers, system services
↓
⑦ OS Ready — login screen / desktop appears

Key Terms Defined

- **RAM (Primary Memory)** — Fast, temporary; erased on power-off. OS runs HERE
- **Disk (Secondary Memory)** — Slow, permanent storage. OS is installed HERE, runs from RAM
- **BIOS/UEFI (Firmware)** — Software burned permanently onto a motherboard chip. Survives power-off. UEFI is modern replacement for BIOS — supports larger disks and graphical interface
- **POST** — Hardware health check. Critical failure = computer won't boot
- **Bootloader** — One job only: find kernel on disk, copy it to RAM, hand control over, then STOP
- **Kernel** — Core of the OS. Highest privilege. Directly controls CPU, RAM, and all hardware
- **Cold Boot** = starting from completely OFF. **Warm Boot** = restart without cutting power
- **Firmware vs Software** — Firmware (BIOS/UEFI) is burned on a chip, rarely changes. Software (OS, apps) lives on disk and updates often

■ EXAM KEY POINTS

- Bootloader's ONLY job = load kernel. It does NOT keep running after kernel takes over
- BIOS/UEFI lives on MOTHERBOARD CHIP — not on the disk. That's why it runs before anything is loaded
- UEFI is modern replacement for BIOS — supports disks > 2TB, graphical interface, faster boot
- Kernel = highest privilege level — it is THE core of the OS (not the whole OS, just the core)
- Cold Boot vs Warm Boot — Cold = from OFF. Warm = restart (Ctrl+Alt+Del) — RAM not fully cleared
- POST failure with critical hardware = computer will not boot at all

TOPIC 02

Process Management

Process States · PCB · CPU Scheduling Algorithms

■ PROCESS — WHAT IS IT?

A **process** = a program **in execution**. A program (e.g., Chrome.exe) sitting on the disk is just a file. The moment the OS loads it into RAM and the CPU starts executing it — it becomes a **process** with its own memory, state, and identity.

Chef Kitchen Analogy

- Recipe book on shelf = Program (just a file, not doing anything)
- Chef actively cooking from that recipe = Process (alive, using resources)
- Single chef (CPU) can only cook ONE order at a time
- Many orders waiting = many processes in the Ready queue
- OS (head chef / manager) decides which order gets cooked next

■ 5 PROCESS STATES + TRANSITIONS

State	Meaning	Transition INTO this state
New	Process is being created by OS	Program launched by user / another process
Ready	Loaded in RAM, waiting for CPU — fully prepared, just needs CPU time	From New (admitted) OR from Waiting (I/O done) OR from Running (preempted)
Running	CPU is actively executing its instructions RIGHT NOW	From Ready — scheduler picks it and allocates CPU
Waiting	Paused — waiting for I/O or an external event. CPU is useless to it right now	From Running — process requested disk read, keyboard input, network data, etc.
Terminated	Finished execution or was killed. Being cleaned up	From Running — process completed OR was force-killed

New → Ready (admitted, loaded into RAM)
↓
Ready → Running (scheduler dispatches, CPU allocated)
↓
Running → Waiting (needs I/O — file read, keyboard, disk — gives up CPU)
↓
Waiting → Ready (I/O completed, back in line for CPU)
↓
Running → Ready (time quantum expired OR higher-priority process arrives)
↓
Running → Terminated (process finishes or is killed)

■ CRITICAL: Ready vs Waiting

- Ready = waiting ONLY for the CPU. Everything else is prepared. Give it the CPU → it runs!
- Waiting = waiting for something OTHER than CPU (disk, keyboard, network). Even if CPU is free, it can't proceed
- Example: Process reading a file → moves to WAITING (disk is slow). Once file is read → back to READY

■ PCB — PROCESS CONTROL BLOCK

The OS creates one **PCB** for every process. It's like the process's **ID card + bookmark combined** — storing everything needed to pause it and resume it later correctly.

What's Inside a PCB?

- **Process ID (PID)** — Unique number (like a token/ticket number) identifying this process
- **Process State** — Current state: New / Ready / Running / Waiting / Terminated
- **Program Counter** — Address of the NEXT instruction to execute. Like a bookmark in a book — 'I was on page 47'
- **CPU Registers** — Small fast storage INSIDE the CPU holding current working values (like a chef's hands mid-chop). Must be saved when interrupted!
- **Memory Info** — Which parts of RAM this process is currently using
- **I/O Status** — Which devices (disk, keyboard, printer) this process is currently using
- **Priority** — Used by scheduling algorithms to decide order of execution

Context Switch = the act of the CPU: (1) saving current process's PCB, (2) loading another process's PCB, (3) resuming execution of that other process. **Pure overhead** — no useful computation happens during the switch itself. Too many context switches = wasted time.

■ CPU SCHEDULING ALGORITHMS

The **scheduler** (part of kernel) decides which Ready process gets the CPU next. Different algorithms make different tradeoffs:

Algorithm	How it Works	Problem / Weakness	Key Term
FCFS (First Come First Served)	Processes run in exact arrival order — like a queue at a ticket counter	Convoy Effect: If a LONG process arrives first, all short ones behind it wait unnecessarily long	Convoy Effect
SJF (Shortest Job First)	Process with shortest CPU burst time runs first	Starvation: A long process may NEVER get CPU if shorter ones keep arriving endlessly	Starvation
Priority Scheduling	Highest-priority process runs first (lower number = higher priority in some systems)	Starvation of low-priority processes. Fixed by AGING: gradually increase priority of waiting processes	Aging
Round Robin (RR)	Each process gets a fixed TIME QUANTUM (e.g. 4ms). If not done, it's paused and moved to back of queue	If quantum too small → too many context switches (overhead). If quantum too large → behaves like FCFS	Time Quantum

Round Robin with Very Large Quantum = FCFS

- If quantum > any process's total CPU time → process always finishes before quantum expires
- → No preemption ever triggers → processes run to completion in arrival order
- → This IS FCFS behaviour! RR becomes FCFS when quantum is huge
- REMEMBER: Round Robin is the basis of all time-sharing / multitasking systems

■ EXAM KEY POINTS

- Preemptive = OS CAN forcibly take CPU away mid-execution (RR, Priority-preemptive, SJF-preemptive)
- Non-preemptive = process keeps CPU until it finishes or voluntarily gives up (FCFS, basic SJF)
- FCFS → Convoy Effect | SJF & Priority → Starvation | RR → quantum-size tradeoff
- Aging = gradual priority boost for waiting processes — prevents starvation in Priority scheduling
- Turnaround Time = Completion Time – Arrival Time (total time in system)
- Waiting Time = Turnaround Time – Burst Time (time spent waiting in Ready queue)
- Response Time = time from arrival until process FIRST gets the CPU
- Context switch = save PCB + load new PCB = pure overhead, no useful work done

TOPIC 03

IPC & Process Synchronization

Race Condition · Critical Section · Semaphores · Mutex

■ INTER-PROCESS COMMUNICATION (IPC)

Processes often need to **talk to each other** or **share data**. IPC is the set of mechanisms that make this possible. Two main approaches:

Shared Memory
• OS sets aside a region of RAM that multiple processes can read/write directly
• FAST — no OS involvement needed for each read/write
• BUT: processes must coordinate themselves to avoid conflicts
• Like a shared whiteboard — anyone can write/erase, but two people grabbing the marker at once = chaos

Message Passing
• Processes communicate by sending/receiving messages THROUGH the OS
• SLOWER — each message goes through the OS (overhead)
• SAFER — OS handles delivery, no shared memory region to corrupt
• Like a mailbox system — write a letter, office peon (OS) delivers it. No direct contact

■ RACE CONDITION — THE CORE PROBLEM

A **race condition** occurs when multiple processes access shared data concurrently, and the **final result depends on the exact timing/order** of their execution — leading to incorrect results if the interleaving is unlucky.

Classic Example: counter = counter + 1
• This ONE line of code actually takes THREE CPU steps:
• Step 1: READ counter value from RAM into CPU register
• Step 2: ADD 1 to the register value
• Step 3: WRITE the register value back to RAM
•
• PROBLEM — Bad Interleaving (counter starts at 5):
• → Process A: Step 1 (reads 5 into its register)
• → Process B: Step 1 (reads 5 into ITS register) ← B reads BEFORE A writes back!
• → Process A: Step 2 (register = 6), Step 3 (writes 6 to RAM)
• → Process B: Step 2 (register = 6), Step 3 (writes 6 to RAM)
• RESULT: counter = 6 (WRONG — should be 7! One increment was LOST)
•
• If B had read AFTER A's write: B reads 6, adds 1 = 7, writes 7 → CORRECT ✓
• The result depends on TIMING = Race Condition

■ CRITICAL SECTION PROBLEM

The **Critical Section** is the part of a process's code that accesses shared resources. We need rules so only ONE process is in its critical section at a time.

Condition	What it Means	Analogy (Washroom)
Mutual Exclusion	Only ONE process can be in the critical section at a time	Only one person inside the washroom at a time — door is locked
Progress	If no one is in the critical section, a waiting process must be allowed in — no unnecessary blocking	If washroom is empty, waiting person should be allowed in immediately — door can't stay locked for no reason
Bounded Waiting	No process should wait FOREVER — there's a limit on how many times others can enter before it gets a turn (anti-starvation)	No one person should be skipped indefinitely while others cut the queue in front

■ Mutual Exclusion ≠ Bounded Waiting

- Mutual Exclusion = only one at a time (WHO gets in = anyone with the lock)
- Bounded Waiting = fair turns — you won't be skipped FOREVER (prevents starvation within critical section)
- You need BOTH conditions satisfied for a correct critical section solution

■ SEMAPHORES & MUTEX

A **Semaphore** is a special integer variable used to control access to shared resources. It has only two operations:

- **wait() — also called P() or down()**: Decreases the semaphore value. If value becomes negative/zero, the process BLOCKS (enters Waiting state) until signalled
- **signal() — also called V() or up()**: Increases the semaphore value. If any process was blocked, it wakes one up

Binary Semaphore (Mutex)

- Value is ONLY 0 or 1
- 0 = LOCKED (someone is inside critical section)
- 1 = UNLOCKED (free to enter)
- Used for a SINGLE shared resource
- Like a washroom lock — either locked (0) or free (1)
- wait() to lock, signal() to unlock

Counting Semaphore

- Value can be ANY non-negative integer
- Value = number of available instances of a resource
- Example: 3 printers → semaphore starts at 3
- Each process using a printer does wait() → value drops
- When done, signal() → value goes back up
- When value = 0, next process blocks

Producer-Consumer Problem

- A classic synchronization problem — know the name and concept!
- Producer process creates data and places it into a shared BUFFER (fixed-size temporary storage)
- Consumer process removes data from the buffer and processes it
- Problem: Producer must not add to a FULL buffer. Consumer must not remove from an EMPTY buffer
- Both must not access the buffer SIMULTANEOUSLY (race condition!)
- Solution: Use semaphores — one counting (tracks slots), one binary (mutual exclusion on buffer)
- This appears frequently in C-CAT MCQs by name — remember: Producer + Consumer + Buffer + Semaphores

■ EXAM KEY POINTS

- Shared Memory = faster, processes coordinate. Message Passing = safer, OS coordinates
- Race Condition = timing-dependent wrong result. Fix = enforce Mutual Exclusion on critical section
- Critical Section needs ALL THREE: Mutual Exclusion + Progress + Bounded Waiting
- wait() = acquire / possibly block | signal() = release / possibly unblock (direction often tested!)
- Binary Semaphore (Mutex) = 0 or 1 = single resource lock
- Counting Semaphore = integer = multiple instances (e.g. 3 printers)
- Busy Waiting (Spinlock) = process loops checking 'are you done?' = wastes CPU cycles
- Proper semaphore = blocking (process enters WAITING state instead of looping)
- Producer-Consumer = classic IPC problem. Know: Buffer + 2 semaphores pattern

TOPIC 04

Deadlock & Deadlock Handling Methods

■ WHAT IS A DEADLOCK?

A **deadlock** is a situation where **two or more processes are permanently stuck waiting for each other** — each holds a resource the other needs, and no one will release until they get what they're waiting for. The system is frozen.

Library Analogy

- Student A holds Book 1 → waiting for Book 2 (held by Student B)
- Student B holds Book 2 → waiting for Book 1 (held by Student A)
- Neither will put their book down until they get the other one
- Neither can ever get the other one → DEADLOCK — both stuck forever
- Key: each is holding something the other needs, and neither will give up what they have

■ 4 COFFMAN CONDITIONS — ALL MUST HOLD SIMULTANEOUSLY

For deadlock to occur, **ALL FOUR** of these conditions must be true at the same time. Break even ONE → no deadlock possible.

Condition	What it Means	Library Analogy	How to BREAK it
1. Mutual Exclusion	At least one resource can only be used by ONE process at a time (non-shareable)	Only one student can hold a book at a time	Make resources shareable (not always possible — e.g. printers can't be shared mid-job)
2. Hold and Wait	A process is HOLDING at least one resource AND WAITING to get more resources held by others	Student A is HOLDING Book 1 WHILE WAITING for Book 2	Force processes to request ALL resources upfront before starting (all-or-nothing)
3. No Preemption	Resources cannot be FORCIBLY taken away — a process holds them until it voluntarily releases	Neither student will give up their book until they finish with it	Allow OS to FORCIBLY take resources away (preemption) — risky but breaks deadlock
4. Circular Wait	There is a CYCLE: A waits for B, B waits for A (or longer: A→B→C→A)	A waiting for B's book, B waiting for A's book — a circular dependency	Impose ordering on resources — processes must request in increasing order number

■ Most Tested: Circular Wait ≠ just 'waiting'

- Circular Wait specifically requires a CYCLE in the waiting chain
- A waiting for B is NOT circular wait alone — B must also be waiting for A (or via C, D...)
- Resource Allocation Graph: draw circles (processes) and squares (resources). If you can trace a cycle following arrows → deadlock!

■ 4 DEADLOCK HANDLING STRATEGIES

Strategy	When it Acts	How it Works	Example / Algorithm
Prevention	Before deadlock can EVER form (design time / policy)	Design the system so at least ONE Coffman condition is STRUCTURALLY IMPOSSIBLE	Force all resources requested upfront (breaks Hold & Wait) OR impose resource ordering (breaks Circular Wait)
Avoidance	Before EACH resource request is granted (runtime check)	Before granting a request, OS checks: 'Will granting this keep the system in a SAFE STATE?' If yes → grant. If no → make process wait	**Banker's Algorithm** — most famous deadlock avoidance algorithm (see below)

Strategy	When it Acts	How it Works	Example / Algorithm
Detection + Recovery	After a deadlock HAS already occurred	Periodically scan for cycles in the Resource Allocation Graph. If deadlock found → recover	Recovery: (1) Kill one deadlocked process OR (2) Forcibly preempt a resource from one process
Ignorance	Never — just pretend the problem doesn't exist	Assume deadlocks are so rare that the overhead of prevention/detection isn't worth it	**Ostrich Algorithm** — surprisingly, used by many real desktop OSes (Windows, Linux)!

Banker's Algorithm — Deadlock AVOIDANCE

• Named after a bank manager who decides whether to grant loans
• ANALOGY: A bank has limited cash (resources). Before giving a loan (resource), the manager asks:
• 'If I give this loan, will I still have enough cash to eventually satisfy ALL other customers?'
• If YES → grant the loan (safe state maintained)
• If NO → refuse for now (even if the customer has valid needs) — wait until enough cash is available
•
• In OS terms:
• Resources = cash | Processes = customers | Max need = maximum loan each customer might need
• SAFE STATE = OS can guarantee every process will eventually finish (even in worst case)
• UNSAFE STATE = doesn't mean deadlock NOW, but deadlock COULD happen → Banker refuses request
• Banker's Algorithm calculates whether granting a request keeps the system in a safe state
• IMPORTANT: Banker's = AVOIDANCE (runtime check per request) NOT Prevention (no structural rules)

■ Most Confused Pair: Prevention vs Avoidance vs Banker's

• Prevention = DESIGN TIME rules. Conditions are made structurally impossible. No runtime check needed
• Avoidance = RUNTIME check before each grant. Conditions CAN occur but we check before allowing
• Banker's Algorithm = specific AVOIDANCE algorithm. NOT prevention!
• Detection = AFTER deadlock forms. Find it, then fix it
• Ignorance (Ostrich) = do nothing. Used in practice for general-purpose OSes

■ EXAM KEY POINTS

• ALL 4 Coffman conditions must hold simultaneously for deadlock
• Breaking even ONE condition = deadlock impossible (Prevention strategy)
• Banker's Algorithm = AVOIDANCE (not prevention!) — most commonly confused pair in C-CAT
• Safe State = all processes can eventually finish. Unsafe ≠ deadlock yet, but could lead to one
• Detection uses Resource Allocation Graph — cycle in graph = deadlock
• Recovery: kill a process OR preempt (forcibly take) a resource
• Ostrich Algorithm = ignore = used by real OSes for general-purpose computing
• Deadlock vs Starvation: Deadlock = frozen cycle. Starvation = unfairly skipped but not cyclically stuck
• Livelock = processes keep responding to each other but make NO real progress (neither frozen nor succeeding)

TOPIC 05

Memory Management

Swapping · Paging · Segmentation · Fragmentation

■ WHY MEMORY MANAGEMENT?

RAM is **fast but limited and expensive**. Multiple processes need RAM simultaneously (from Topic 2 — many processes in Ready/Running state). The OS must allocate, track, and free RAM efficiently while protecting each process's space from others.

■ SWAPPING

Swapping = temporarily moving an **entire process** out of RAM to a reserved disk area called **Swap Space**, freeing up RAM for other processes. When the swapped-out process is needed again, it is moved back into RAM.

When and Why Swapping Happens

- Scenario: RAM is full. New process needs to run OR waiting process needs to resume
- OS picks a process currently not active (e.g. in Waiting state) and swaps it OUT to disk
- Its RAM space is freed → used for the new/needed process
- Later: swapped-out process is moved back IN from disk to RAM to resume
- SLOW: moving entire processes to/from disk takes significant time (disk is much slower than RAM)
- Swap Space = a special reserved area on disk just for this purpose — not regular file storage

■ CONTIGUOUS MEMORY ALLOCATION — THE OLD WAY (sets up WHY Paging exists)

Early approach: give each process one **single continuous block of RAM**. As processes come and go, RAM ends up looking like:

Fragmentation Problem

- RAM after several processes: [Process A: 10MB][FREE: 2MB][Process B: 15MB][FREE: 3MB][Process C: 8MB]
- New process needs 4MB. Available free blocks: 2MB and 3MB
- Neither block is big enough individually — even though 5MB TOTAL is free!
- This is EXTERNAL FRAGMENTATION: free memory exists but is scattered in unusable small pieces
- Fix: use PAGING (below) — which doesn't require contiguous allocation

■ PAGING — THE SOLUTION TO EXTERNAL FRAGMENTATION

Paging = divide BOTH the process's memory AND the RAM into small, equal-sized blocks. Any block of the process can go into ANY free block of RAM — no need for them to be next to each other.

Term	Definition	Example
Page	A fixed-size chunk of a PROCESS'S memory	Process divided into Page 0 (4KB), Page 1 (4KB), Page 2 (4KB)...
Frame	A fixed-size chunk of RAM — SAME size as a page	RAM divided into Frame 0, Frame 1, Frame 2... each 4KB
Page Table	Per-process lookup table maintained by OS: maps each Page → its Frame in RAM	Page 0 → Frame 5, Page 1 → Frame 12, Page 2 → Frame 3

Process's memory divided into equal PAGES (e.g. 4KB each)
↓
RAM divided into equal FRAMES (same 4KB size)
↓
OS places each PAGE into any available free FRAME (not necessarily adjacent!)
↓
Page Table records: Page 0 → Frame 5, Page 1 → Frame 12, Page 2 → Frame 3
↓
CPU needs to access Page 2? Looks up Page Table → goes to Frame 3 in RAM

External Fragmentation
- Free space exists but is SCATTERED in small gaps between allocated blocks
- Caused by: Contiguous Allocation or Segmentation
- Solved by: Paging (any free frame works for any page)
- Think: gaps between books on a shelf — individually too small for a new book

Internal Fragmentation
- Wasted space INSIDE an allocated block — block not fully used
- Caused by: Paging (fixed block sizes)
- Example: Page size = 4KB. Last page only needs 1KB → 3KB inside the frame is WASTED
- Cannot be given to another process — that frame is reserved for this page

■ SEGMENTATION

Segmentation = divide a process's memory based on **logical meaning** — Code, Data, Stack segments — each is a different size (unlike paging's fixed sizes).

Paging
- Fixed equal sizes (e.g. all 4KB)
- No relation to program structure
- Page 1 might contain part of code AND part of data — doesn't care
- Tracks with Page Table (page → frame)
- No external fragmentation, but internal fragmentation
- Machine-oriented (hardware convenience)

Segmentation
- Variable sizes — each segment is as big as it needs to be
- Based on LOGICAL program structure (Code / Data / Stack)
- Code segment = all instructions. Data segment = global variables. Stack segment = function calls
- Tracks with Segment Table (segment → base address + length)
- Can have external fragmentation (variable sizes), no internal fragmentation
- Programmer-oriented (logical convenience)

■ EXAM KEY POINTS

- Page = piece of PROCESS. Frame = piece of RAM. Both same fixed size
- Page Table = per-process map: Page Number → Frame Number (OS maintains this)
- Paging SOLVES External Fragmentation BUT INTRODUCES Internal Fragmentation
- External Frag = scattered free space (contiguous/segmentation problem). Internal Frag = wasted space inside block (paging problem)
- Segmentation = variable-size logical units (Code/Data/Stack) with Segment Table storing base + length
- Contiguous Allocation → External Fragmentation → fixed by Paging
- Swapping = slow (whole process to disk). Only used when RAM is full

TOPIC 06

Virtual Memory, Demand Paging & Thrashing

■ VIRTUAL MEMORY — THE BIG IDEA

Virtual Memory = a technique that gives each process the **illusion of having a large, private memory space**, even if the total RAM available is smaller than the process's total needs. It works by keeping only the **currently-needed pages** in RAM, and storing the rest on disk.

Study Desk Analogy

- Library (all books) = process's total memory requirements (could be huge)
- Your study desk = RAM (small, limited space)
- You don't bring ALL books to your desk — just the ones you need RIGHT NOW
- When you need a different book, go to the shelf (disk), fetch it, bring it to desk
- If desk is full, put back a book you're not currently reading to make space
- From your perspective: you have access to the ENTIRE library at all times (illusion!)
- Benefit: processes can be larger than physical RAM — OS manages the movement automatically

■ DEMAND PAGING — HOW VIRTUAL MEMORY IS IMPLEMENTED

Demand Paging = pages are loaded into RAM **only when the process actually needs (demands) them**. NOT pre-loaded in advance. This is the standard implementation of virtual memory.

Process starts — initially very few (or zero) pages in RAM. Page Table entries point to disk
↓
CPU tries to access a memory address (for an instruction or data)
↓
Hardware checks Page Table: Is this page currently in a RAM frame?
↓
PAGE HIT: Page IS in RAM → access proceeds immediately (fast ✓)
↓
PAGE FAULT: Page is NOT in RAM → triggers OS intervention (slow!)
↓
Process moves to WAITING state (I/O needed — disk read) ← connects to Topic 2 states!
↓
OS finds a free RAM frame (or evicts one — see Page Replacement below)
↓
OS reads the required page from disk into the free frame (disk I/O happens here)
↓
Page Table updated: this page now mapped to the new frame
↓
Process moves back to READY, then RUNNING. The instruction is retried — now it's a Page Hit

■ Page Fault Cost

- A Page Fault requires a DISK READ — which is orders of magnitude slower than RAM access
- RAM access: ~nanoseconds. Disk access: ~milliseconds. Disk is 100,000x+ slower!
- Even a small page fault rate significantly slows the system
- Goal: minimize page faults by keeping the RIGHT pages in RAM

■ PAGE REPLACEMENT ALGORITHMS

When a Page Fault occurs and RAM has **NO free frames** (all occupied), OS must **EVICT one existing page** to make room. Which page to evict?

Algorithm	Eviction Rule	Advantage	Disadvantage
FIFO (First In First Out)	Remove the page that has been in RAM the LONGEST (oldest arrival)	Simple to implement — just track when each page was loaded	Ignores actual usage — may evict a page that's been in RAM long but is still frequently used. Poor performance
LRU (Least Recently Used)	Remove the page that has NOT been USED for the longest time	Better performance — recently-used pages are likely to be used again soon (locality principle)	Harder to implement — must track time of last use for every page
Optimal	Remove the page that won't be needed for the LONGEST TIME in the future	Theoretically the best possible — fewest page faults	IMPOSSIBLE in practice — requires knowing the future. Used only as a benchmark to compare other algorithms against

■ Belady's Anomaly — FIFO Only!

- You'd expect: MORE frames in RAM → FEWER page faults. This is normally true
- Belady's Anomaly: With FIFO specifically, adding MORE frames can sometimes INCREASE page faults!
- This is counterintuitive and is a famous exam trick
- LRU and Optimal do NOT suffer from Belady's Anomaly — only FIFO!

■ THRASHING — WHEN VIRTUAL MEMORY GOES WRONG

Thrashing = a severe performance collapse where the system spends most of its time **swapping pages in and out of disk** (handling page faults) and almost no time actually **executing process instructions**.

Too many processes running → each process gets very few RAM frames
↓
Each process constantly needs pages NOT in its small frame allocation → frequent Page Faults
↓
Each Page Fault → process moves to WAITING state while disk fetch happens
↓
CPU sits IDLE waiting for page faults to resolve → CPU utilization DROPS
↓
Disk is working overtime → high disk activity but almost zero actual computation
↓
OS notices low CPU utilization and (WRONG response) adds MORE processes to 'keep CPU busy'
↓
More processes → even fewer frames per process → MORE page faults → WORSE thrashing!

How to FIX Thrashing

- CORRECT FIX: REDUCE the number of running processes → each remaining process gets MORE frames
- More frames per process → fewer page faults → process actually runs → CPU utilization recovers
- Alternative fix: Add MORE physical RAM to the system
- Working Set Model: Track how many pages each process actively uses (its working set) and ensure it gets at least that many frames
- WRONG response: Adding MORE processes when CPU utilization is low due to thrashing makes it WORSE

■ EXAM KEY POINTS

- Virtual Memory = illusion of large memory using RAM + disk together
- Demand Paging = load page ONLY when accessed (not in advance) — standard VM implementation
- Page Fault = page not in RAM → disk fetch needed → process goes to WAITING state
- Page Hit = page found in RAM → access proceeds normally (fast)

- FIFO: evict oldest-loaded. LRU: evict least-recently-used. Optimal: theoretical best only
- Belady's Anomaly = FIFO only — more frames can = MORE page faults (counterintuitive!)
- LRU and Optimal do NOT have Belady's Anomaly
- Thrashing = CPU utilization DOWN + disk activity UP + barely any execution
- Fix thrashing = REDUCE processes (not add more!) or add RAM
- Effective Access Time = (1-p) × RAM_time + p × Disk_time (p = page fault rate)

TOPIC 07

System Calls, Dual Mode Operation, Kernel & User Mode

■ WHY DUAL MODE?

If every user program had unrestricted access to hardware (CPU, memory, disk), one buggy or malicious program could: overwrite another process's memory, corrupt the file system, or crash the entire computer. The OS protects against this using **Dual Mode Operation**.

Bank Vault Analogy

- Customer Area = User Mode (where regular programs run — limited access)
- Secure Vault Room = Kernel Mode (where OS kernel runs — full access to everything)
- Customers CANNOT enter the vault directly — they lack the key (privilege)
- To get something from the vault, a customer fills a REQUEST SLIP (system call)
- Bank Teller (Kernel) takes the slip, enters the vault, performs the action, returns the result
- This way: customers get what they need WITHOUT ever having direct vault access
- Protection: even if a customer goes crazy, they can't damage the vault contents

■ USER MODE vs KERNEL MODE

Aspect	User Mode (Mode Bit = 1)	Kernel Mode (Mode Bit = 0)
Who runs here?	Regular applications — browser, games, text editor, your code	OS Kernel — the core of the OS (from Topic 1)
Hardware access?	NO — completely restricted from direct hardware access	YES — full, direct access to all hardware
Privileged instructions?	BLOCKED — if attempted, CPU raises an error/trap and likely kills the program	ALLOWED — can execute any CPU instruction including hardware-control ones
Memory access?	Limited to the process's own assigned memory space only	Can access ANY memory location in the entire system
Who can switch TO this mode?	Only by making a system call or triggering a hardware interrupt/trap	Kernel can return to User Mode after completing a system call

The Mode Bit — Hardware-Level Switch

- A SINGLE bit inside the CPU hardware that indicates the current operating mode
- Mode Bit = 1 → User Mode (restricted)
- Mode Bit = 0 → Kernel Mode (full privilege)
- The CPU checks this bit BEFORE executing any instruction
- If it's a privileged instruction and Mode Bit = 1 → CPU BLOCKS IT immediately and raises a TRAP
- The mode bit can only be changed by the OS kernel or hardware events (not by user programs!)

■ SYSTEM CALLS — THE CONTROLLED GATEWAY

A **system call** is the mechanism by which a User Mode program **formally requests the kernel** to perform a privileged operation on its behalf. It is the ONLY safe, controlled path into Kernel Mode.

User Mode program needs privileged operation (e.g., READ a file from disk)
↓
Program executes a special system call instruction (also called a 'trap' or 'software interrupt')
↓
CPU detects system call → MODE SWITCH: Mode Bit flips 1→0 (User → Kernel Mode)
↓
CPU now in Kernel Mode — kernel code runs with full privileges
↓
Kernel validates the request, then performs the privileged operation (disk read, memory alloc, etc.)
↓
Kernel places result in a location the user program can access
↓
MODE SWITCH BACK: Mode Bit flips 0→1 (Kernel → User Mode)
↓
User program resumes execution with the result of its request

■ SYSTEM CALL CATEGORIES

Category	Purpose	Examples
Process Control	Create, run, wait for, or kill processes	fork(), exec(), exit(), wait() — connects to Topic 2 process states!
File Management	Open, read, write, close, delete files	open(), read(), write(), close(), unlink() — connects to Topic 8!
Device Management	Request, release, and use hardware devices	ioctl(), read(), write() on device files
Information Maintenance	Get/set system info, time, process details	getpid(), time(), alarm()
Communication	Send/receive messages between processes	pipe(), socket(), send(), recv() — connects to Topic 3 IPC!

Trap vs Interrupt — Know the Difference

• **Trap (Software Interrupt)** = triggered BY the program intentionally (system calls) or due to a program error (division by zero, invalid memory access). Process-generated
• **Hardware Interrupt** = triggered BY HARDWARE to notify CPU of an event (disk finished reading, keyboard key pressed, network packet arrived). Hardware-generated
• **Page Fault** (Topic 6) = HARDWARE-triggered trap — CPU detects page not in RAM and automatically switches to Kernel Mode to handle it. NOT a deliberate system call by the program!
• All three (system call, trap, interrupt) result in a MODE SWITCH to Kernel Mode

■ EXAM KEY POINTS

- Dual Mode = User Mode (restricted, Mode Bit=1) + Kernel Mode (full privilege, Mode Bit=0)
- Mode Bit checked by CPU before EVERY privileged instruction
- System Call = ONLY controlled gateway from User Mode to Kernel Mode
- Attempted privileged instruction in User Mode → CPU blocks it → raises trap → usually kills program
- Page Fault = HARDWARE triggers mode switch automatically (not a deliberate system call!)
- Trap = software-generated mode switch | Interrupt = hardware-generated mode switch
- Both traps and interrupts cause mode switch to Kernel Mode
- System Call categories: Process Control, File Mgmt, Device Mgmt, Info Maintenance, Communication
- Mode switching has overhead (like context switching) — too many system calls can slow a program

TOPIC 08

File & Storage Management — File System Structure

■ FILE AND FILE SYSTEM — DEFINITIONS

A **File** = a named, organized collection of related data stored on secondary memory (disk — permanent storage that survives power-off, unlike RAM). Files can be programs, documents, images, videos, etc.

A **File System** = the OS component that manages HOW files are named, organized, stored, and retrieved on disk. It is the **catalog/organizer** — it tracks WHERE data is, not the data itself.

Library Catalog Analogy

- Each Book = a File (unit of stored information with a name)
- Library Catalog System = File System (tells you WHERE book X is — Shelf 5, Row 2)
- The catalog does NOT contain the books' content — just their locations and details
- Library sections (Fiction / Science) = Directories/Folders
- Sections within sections = sub-directories (Tree structure)

■ FILE ATTRIBUTES (METADATA)

Metadata = 'data about data' — information DESCRIBING a file, stored separately from the file's actual content. When you right-click a file and see Properties, you're seeing its metadata.

Attribute	What it Is	Example
Name	Human-readable identifier	report.docx, photo.jpg, main.py
Size	Amount of data the file contains	2.5 MB, 750 KB
Type	Kind of file — often shown via extension	.txt (text), .exe (executable), .jpg (image)
Location	Where on disk the data physically lives	Block numbers on the disk
Permissions	Who can do what with this file	Read (R), Write (W), Execute (X) for owner/group/others
Timestamps	Time-related tracking	Created, Last Modified, Last Accessed

■ FILE OPERATIONS — VIA SYSTEM CALLS (Topic 7 connection!)

All file operations go through **system calls** (Topic 7) because disk access is a privileged operation requiring Kernel Mode:

- **Create** — allocate space on disk, create an entry in the directory
- **Open** — load file metadata, prepare for access (required before read/write)
- **Read** — fetch file content from disk blocks into RAM for the process to use
- **Write** — save data from RAM back to disk blocks
- **Seek** — move to a specific position within the file (for Direct Access)
- **Close** — finish using file, release resources, ensure data is written to disk
- **Delete** — remove directory entry, mark disk blocks as free

■ DIRECTORY STRUCTURES

A **Directory** = a special type of file that stores a **list of entries** pointing to other files and sub-directories. It does NOT contain user data — it contains pointers/references telling the OS WHERE to find other files.

Single-Level Directory: ALL files in ONE flat list. No folders at all
↓
Problem: No two files can share the same name. Chaos with many files
↓
Two-Level Directory: Each USER gets their own separate directory
↓
Fix: User A and User B can both have 'notes.txt' (in different user directories)
↓
Problem: No organisation WITHIN a user's files — still a flat list per user
↓
Tree-Structured Directory: Directories contain files AND sub-directories
↓
Most common in all modern OSes — folders inside folders inside folders
↓
No naming conflicts (same name in different folders is fine)
↓
PATH = route from root through directories to the file

PATH — Route Through the Directory Tree

- Path = string listing each directory you pass through to reach a file
- Separator: '/' on Linux/Mac, '\\' on Windows
- Example: /Users/Alice/Documents/report.docx
- / = root directory → Users folder → Alice's subfolder → Documents subfolder → file
- Absolute path = starts from root (/ or C:\\) — always works regardless of current location
- Relative path = starts from current directory (e.g. Documents/report.docx if already in Alice/)

■ FILE ACCESS METHODS

Sequential Access

- Read data IN ORDER from start to end
- Cannot skip ahead — must pass through all preceding data
- Like reading a book page by page — can't jump to page 99 without reading 1-98
- Best for: log files, audio/video streaming, reading entire file
- Simple but inefficient if you only need a small part in the middle/end

Direct (Random) Access

- Jump DIRECTLY to any position using an offset or block number
- Like opening a book directly to page 99 using the page number
- No need to read preceding data
- Best for: databases, reading last paragraph of large file, any partial access
- Requires knowing the position/offset of the desired data

■ EXAM KEY POINTS

- File = named data on disk. File System = catalog/organizer (NOT the content itself)
- Metadata = data ABOUT a file (name, size, type, permissions, timestamps) — not the file's content
- Directory = stores LIST of entries + pointers — NOT actual user data
- Single-Level → Two-Level → Tree-Structured (most common — know the evolution and problems solved)
- Tree-Structured = modern OS folder system — folders within folders, no naming conflicts
- Path = route from root to file. '/' separates levels. Absolute vs Relative paths
- Sequential = must read in order. Direct/Random = jump anywhere (use for 'read last paragraph')
- Mounting = attaching an external disk's file system into the main directory tree
- Permissions: R (read), W (write), X (execute) — enforced by kernel via system calls (Topic 7 link!)

TOPIC 09

Disk Space Allocation & Disk Scheduling Algorithms

■ SETUP: FILES AND DISK BLOCKS

From Topic 8: a file's data is stored in **disk blocks** (fixed-size chunks of disk — the smallest unit the disk reads/writes). A file spanning multiple blocks needs a system to track which blocks belong to it and in what order.

■ PART A: DISK SPACE ALLOCATION METHODS

Three main methods for allocating disk blocks to files:

Method	How It Works	Random Access?	External Frag?	Key Weakness
Contiguous Allocation	All blocks stored in CONSECUTIVE disk locations. File starts at block X, uses X, X+1, X+2...	YES — easy: block N = start + N (simple arithmetic)	YES — big problem	As files are created/deleted, free space becomes scattered — new large file may not find enough consecutive space
Linked Allocation	Blocks can be ANYWHERE on disk. Each block contains a POINTER to the NEXT block of the same file (chain/linked list)	NO — must follow chain from block 1 to reach block N (must traverse all preceding blocks)	NO — any free block works	No random access. Pointer corruption breaks the chain (can't reach rest of file). Pointer storage overhead in each block
Indexed Allocation	Each file has a dedicated INDEX BLOCK. The index block contains a list of pointers to ALL blocks of the file	YES — look up entry N in index block directly	NO — any free block works	Index block uses extra disk space. Very large files may need multi-level index blocks

Visual: How Each Method Tracks File Data

• CONTIGUOUS: File A = [Block 5][Block 6][Block 7][Block 8] (all next to each other)
•
• LINKED: File A = Block 12 → Block 3 → Block 19 → Block 7 (each block points to next)
• Directory just stores first block address (Block 12). Must follow chain to reach Block 19
•
• INDEXED: Index Block for File A contains: [→Block 12][→Block 3][→Block 19][→Block 7]
• To find block 3 of file: look at Index Block entry 3 → gives address directly
•
• PATTERN ACROSS TOPICS: Contiguous Memory (Topic 5) + Contiguous Disk (Topic 9) → both External Fragmentation
• Paging (Topic 5) + Indexed Allocation (Topic 9) → both use pointer/table structures to solve fragmentation

■ Linked Allocation = Sequential Access ONLY

• To find block 50 of a file: must start at block 1, follow pointer to block 2, then 3... then 49, then 50
• You CANNOT jump directly — there's no way to know where block 50 is without following the chain
• This links back to Topic 8: Linked Allocation forces SEQUENTIAL ACCESS on the file
• Contiguous and Indexed Allocation both support DIRECT ACCESS (can jump to any block directly)

■ PART B: DISK SCHEDULING ALGORITHMS

The disk has a physical **read/write head** that must move across the disk's surface to reach requested data. Multiple processes may request different disk locations simultaneously.

Seek Time = time for the disk head to move to the requested position. This is the main cost disk scheduling minimises.

Setup for All Examples

- Disk tracks numbered: 0 to 199 (like positions 0 to 199)
- Head currently at position: 50
- Pending requests at positions: 98, 183, 37, 122, 14, 124, 65, 67
- Goal: service all requests while minimising total head movement distance

Algorithm	Head Movement Rule	Advantage	Disadvantage	Parallel to CPU Scheduling
FCFS	Service in EXACT arrival order: 50→98→183→37→122→14→124→65→67	Simple, fair in arrival order, no starvation	Head moves erratically back and forth — huge total movement, very inefficient	Same as FCFS in CPU scheduling (arrival order, convoy-effect-like waste)
SSTF (Shortest Seek Time First)	Always service the request CLOSEST to current head position next	Much lower average seek time than FCFS	STARVATION: requests far from the head keep getting skipped as closer ones keep arriving	Like SJF in CPU scheduling (shortest/closest first, same starvation risk)
SCAN (Elevator)	Move head in ONE direction, service all requests in path, reach end, REVERSE, service on way back	No starvation. More uniform than SSTF. Predictable movement	Requests just behind the head's direction get served later (wait for full sweep + return)	No direct CPU scheduling equivalent — elevator concept
C-SCAN (Circular SCAN)	Like SCAN but on reaching one end, head JUMPS back to start WITHOUT servicing on the return. Then sweeps again in same direction	Most UNIFORM wait times across all tracks — near-start tracks don't wait for full reverse cycle	Slight overhead from non-serving return jump — head moves without doing work on return	Like a one-directional elevator that teleports back to ground floor

SCAN

- Head moves in one direction, services all requests
- Reaches END of disk, then REVERSES
- Services requests on the WAY BACK too
- Like elevator: goes up serving all floors, comes back down serving floors
- Slight unfairness: tracks just past the end wait for a full sweep + return

C-SCAN

- Head moves in one direction, services all requests
- Reaches END of disk, then JUMPS back to start (no service during jump)
- Services requests only in ONE direction
- Like elevator that goes up, then teleports to ground floor (no stops going down)
- More UNIFORM: tracks near start don't wait for entire down sweep

■ EXAM KEY POINTS

- Contiguous = fast random access + simple BUT external fragmentation as files created/deleted
- Linked = no fragmentation + flexible BUT no random access (must follow chain)
- Indexed = best of both (random access + no fragmentation) BUT extra index block storage needed
- Large files with Indexed Allocation need MULTI-LEVEL index blocks (like multi-level page tables in Topic 5!)
- Seek Time = time for head to physically MOVE to a track — what disk scheduling minimises
- FCFS (disk) = arrival order, erratic. SSTF = closest first, can starve (like SJF starvation)
- SCAN = elevator (both directions). C-SCAN = circular (one direction + jump) = most uniform
- SSTF starvation mirrors SJF starvation. FCFS disk mirrors FCFS CPU — same patterns!

■ MASTER REVISION CHEATSHEET — ALL 9 TOPICS

Topic	Must-Know Concept	Common Exam TRAP
01 OS & Boot	OS = resource manager. POST→Bootloader→Kernel. Kernel = highest privilege, always in memory.	Bootloader STOPS after loading kernel (doesn't keep running). UEFI ≠ BI
02 Process States	5 states: New→Ready→Running→Waiting→Terminated. PCB stores: PID, state, program counter, registers, memory info.	Ready = waiting for CPU. Waiting = needs I/O (even fre
02 Scheduling	FCFS=arrival order(convoy). SJF=shortest first(starvation). Priority=priority order(starvation). RR=time quantum(fair).	Preemptive fixes starvation. FCFS=simplest(unfair). Context switch=pure overhead. Aging fi
03 IPC	Shared Memory (fast, self-sync). Message Passing (slower, OS-sync). Race condition = unsynchronized access.	wait() = acquire/BLOCK, signal() = release/UNBLOCK. NOT the other wa
03 Sync	Critical Section needs ALL THREE: Mutual Exclusion + Progress + Bounded Waiting.	Bounded Waiting ≠ Mutual Exclusion. BW = anti-starvation guarantee with
04 Deadlock	ALL 4 Coffman conditions must hold: Mutual Exclusion + Hold&Wait + No Preemption + Circular Wait.	Breaking ONE Coffman condition = deadlock impossible. Circular W
04 Handling	Prevention=design rules. Avoidance=Banker's Algorithm (safe state check). Detection=RAG cycle. Ignore=ostrich.	Banker's = AVOIDANCE (runtime check), NOT Prevention (most confuse
05 Paging	Page=process chunk, Frame=RAM chunk (same size). Page Table maps Page→Frame.	Paging SOLVES external frag but INTRODUCES internal frag (wasted sp
05 Fragmentation	External = scattered free space between blocks (contiguous/segmentation). Internal = wasted space inside a block (paging).	Segmentation = variable-size logical units (Code/Data/Stack). Has externa
06 Virtual Mem	Page Fault = page not in RAM → disk fetch → process to WAITING state. Page table lives in RAM (fast).	Belady's Anomaly = FIFO ONLY: more frames can = MORE page faults.
06 Thrashing	Too many processes → too few frames → constant page faults → CPU idle despite high load.	Fix: REDUCE processes (not add more despite low CPU — ac
07 Dual Mode	Mode Bit=1: User Mode (restricted). Mode Bit=0: Kernel Mode (full privilege). System call = HARDWARE triggers mode switch automatically (not a delib	
08 File System	Directory = list of pointers, NOT user data. Tree-Structured = most common. Path = absolute from root.	Metadata ≠ file content. Direct Access = jump anywhere. Sequential = mu
09 Disk Alloc	Contiguous=fast+ext.frag. Linked=no random access+no ext.frag. Indexed=random access+extra index block.	Linked Allocation = SEQUENTIAL ACCESS ONLY — cannot jump to bloc
09 Disk Sched	SCAN=elevator(both ways). C-SCAN=one direction+jump(most uniform). SSTF=closest first(can starve).	SSTF ≈ SJF for disk. FCFS=fair but worst. C-SCAN mirrors FCFS CPU. C-SCAN mo

Phase 1 Complete — Operating Systems: 9 Chapters, Zero to Hero ✓

Next: Phase 2 — Computer Networks (Topics 10–13). You're ready!